



Εθνικό Μετσόβιο Πολυτεχνείο

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Υπολογιστών

Τομέας Τεχνολογίας Πληροφορικής & Υπολογιστών

7^ο Εξάμηνο: Εργαστήριο Λειτουργικών Συστημάτων

2^η Άσκηση:
Οδηγός Ασύρματου Δικτύου Αισθητήρων
στο Λειτουργικό Σύστημα Linux

Ημερομηνία Παρουσίασης: 5/12/2024

Γιάννης Πολυχρονόπουλος

Αριθμός Μητρώου: 03121089

Κωνσταντίνος Σταυλιώτης

Αριθμός Μητρώου: 03121104

linux_chrdev_state_needs_refresh

Η συνάρτηση `linux_chrdev_state_needs_refresh` χρησιμοποιείται για να μας ενημερώσει αν έχει αλλάξει η κατάσταση του αισθητήρα μας από την τελευταία φορά που τον ελέγξαμε. Για παράδειγμα, μπορεί να έχουν έρθει νέα δεδομένα, οπότε πρέπει να ανανεώσουμε τη δομή στην οποία τα καταχωρούμε. Λαμβάνει σαν είσοδο τη δομή `linux_chrdev_state_struct` και επιστρέφει 1 αν η τελευταία ενημέρωση της συσκευής έχει γίνει μετά από το δικό μας timestamp και 0 στην αντίθετη περίπτωση. Το timestamp περιγράφει τη στιγμή που διαβάσαμε τελευταία φορά τον αισθητήρα.

```
static int linux_chrdev_state_needs_refresh(struct linux_chrdev_state_struct *state)
{
    struct linux_sensor_struct *sensor;

    WARN_ON ( !(sensor = state->sensor)); /* Logs a warning message in the kernel
        logs if the condition is true (non zero) */
    /*-----OUR CODE-----*/
    /* Compare the cached timestamp with the sensor's last update */
    if (sensor->msr_data[state->type]->last_update > state->buf_timestamp) {
        return 1; /* Refresh needed
    }
    return 0; /* No refresh needed

    /*-----*/
}
```

linux_chrdev_state_update

Η συνάρτηση `linux_chrdev_state_update` καλείται για την ενημέρωση της δομής `state` της συσκευής μας. Η συνάρτηση αυτή θεωρείται κρίσιμο τμήμα και καλείται με το σημαφόρο του `state` κλειδωμένο (`down_interruptible`), καθώς δε θέλουμε να επιτρέψουμε σε πολλές διεργασίες να αλλάξουν τα δεδομένα του `state` ταυτόχρονα. Για ασφαλή πρόσβαση κλειδώνουμε ξανά το σημαφόρο μέσα στη συνάρτηση αυτή τη φορά (`spin_lock_irqsave`) για να διαβάσουμε τα δεδομένα του αισθητήρα. Πιο συγκεκριμένα, τα δεδομένα περιλαμβάνουν τη χρονική στιγμή κατά την οποία έγινε η τελευταία μέτρηση και την ίδια τη μέτρηση. Η διαφορά από το κλείδωμα `down_interruptible` είναι ότι το πρώτο θα ανοίξει σε περίπτωση κάποιου `interrupt`, ενώ το δεύτερο απενεργοποιεί τις διακοπές στη CPU στην οποία εκτελείται μέχρι να ανοίξει. Πριν ενημερώσουμε το `state`, ελέγχουμε αν χρειάζεται ανανέωση με τη συνάρτηση `linux_chrdev_state_needs_refresh` που περιγράψαμε παραπάνω. Αν διαπιστωθεί ότι το `state` είναι ήδη ενημερωμένο και δε χρειάζεται να προχωρήσουμε. Η συνάρτηση επιστρέφει το σφάλμα `-EAGAIN` που σηματοδοτεί ότι δεν υπάρχουν καινούρια δεδομένα και ο χρήστης πρέπει να την ξανακαλέσει αργότερα. Όταν τελικά έρθουν νέα δεδομένα, χρειάζεται να ενημερώσουμε το `state` της συσκευής. Ενημερώνουμε το `timestamp` `buffer`, μετατρέπουμε τη μέτρηση σύμφωνα με το επιθυμητό `format` και επιβεβαιώνουμε ότι ο αριθμός χωράει στο `buffer`. Τέλος, καλούμε τη συνάρτηση `memcpy` για να περάσουμε τη μέτρηση στη δομή του `state`. Σε περίπτωση επιτυχίας, η συνάρτηση επιστρέφει 0.

```
static int linux_chrdev_state_update(struct linux_chrdev_state_struct *state)
{
    struct linux_sensor_struct __attribute__((unused)) *sensor;
    /*-----OUR CODE-----*/

    struct linux_msr_data_struct *msr_data;
    unsigned long flags; /* For spinlock */
    uint32_t raw_value, last_update;
    char formatted_data[LINUX_CHRDEV_BUFSZ]; /* Buffer for formatted data */
    int formatted_len;
    long converted_value;
    long *lookup[N_LUNIX_MSR] = {lookup_voltage, lookup_temperature, lookup_light};

    /*-----*/
    /* output text to the kernel log buffer for debugging purposes */
    debug("entering\n"); /* Changed it to "entering" because it made a lot more sense
        */
}
```

```

/*
 * Grab the raw data quickly, hold the
 * spinlock for as little as possible.
 */
/*-----OUR CODE-----*/

/* Validate the sensor */
WARN_ON(!(sensor = state->sensor));

    /* Acquire the spinlock to safely access shared sensor data */
    spin_lock_irqsave(&sensor->lock, flags);

/* Fetch the raw data */
msr_data = sensor->msr_data[state->type];
last_update = msr_data->last_update;
raw_value = msr_data->values[0];

/* Unlock */
spin_unlock_irqrestore(&sensor->lock, flags);
/*-----*/

/*
TODO: Why use spinlocks? See LDD3, p. 119
 * Answer: spinlocks protect sensor->msr_data because multiple threads or
interrupts may access and modify the sensor data concurrently.
 * We use irqsave to disable interrupts to prevent deadlocks and we save the
interrupt state to the "flags" variable.
 */

/*
 * Any new data available?
 */
/*-----OUR CODE-----*/

/* Check if the state needs a refresh */
if (!linux_chrdev_state_needs_refresh(state)) {
    debug("No new data available...\n");
    return -EAGAIN; // Error code that means "try again."
}

/*-----*/

/*
 * Now we can take our time to format them,
 * holding only the private state semaphore
 */

/*-----OUR CODE-----*/

/* Update buffer timestamp */
state->buf_timestamp = last_update;

/* Convert and format data */
converted_value = lookup[state->type][raw_value]; // Using the lookup table

    /* The converted value is in the form XXYYY while we want XX.YYY. For XX we
get it from /1000 (int division) and the YYY from %1000 */
formatted_len = snprintf(formatted_data, LINUX_CHRDEV_BUFSZ, "%ld.%03ld\n",
    converted_value / 1000, converted_value % 1000);

/* Check for buffer overflow */
if (formatted_len >= LINUX_CHRDEV_BUFSZ) {
    return -EOVERFLOW; // An error code defined in Linux kernel headers,
    representing Value Too Large for Defined Data Type.
}

```

```

    /* Update the state buffer */
    memcpy(state->buf_data, formatted_data, formatted_len); // Copy the data to the
    buffer.
    state->buf_lim = formatted_len;

    /*-----*/

    debug("leaving\n");
    return 0;
}

```

linux_chrdev_open

Εισαγωγή:

Η συνάρτηση `linux_chrdev_open` αποτελεί τη βασική συνάρτηση που εκτελείται όταν ένας χρήστης ή μια εφαρμογή ανοίγει μια συσκευή Linux μέσω της κλήσης συστήματος `open()`. Η συνάρτηση καλείται από τον πυρήνα Linux μέσω της δομής `file_operations` που ορίζει τις συναρτήσεις για τη συσκευή. Είναι υπεύθυνη για την αρχικοποίηση της κατάστασης του χαρακτήρα συσκευής (δομή που δίνεται στον παρακάτω κώδικα) και τη σύνδεσή της με τον κατάλληλο αισθητήρα (sensor).

```

struct linux_chrdev_state_struct {
    enum linux_msr_enum type;
    struct linux_sensor_struct *sensor;

    /* A buffer used to hold cached textual info */
    int buf_lim;
    unsigned char buf_data[LINUX_CHRDEV_BUFSZ];
    uint32_t buf_timestamp;

    struct semaphore lock;

    /*
     * Fixme: Any mode settings? e.g. blocking vs. non-blocking
     */
    int mode; /* Blocking (1) or non-blocking (0) mode */
};

```

Ρόλος της Συνάρτησης

Η συνάρτηση εξυπηρετεί δύο κύριες λειτουργίες:

1. Επικύρωση του αιτήματος ανοίγματος:

- Ελέγχει αν ο `minor` αριθμός αντιστοιχεί σε έγκυρο αισθητήρα και τύπο μέτρησης.
- Επιστρέφει σφάλμα αν δεν είναι δυνατή η σύνδεση με τον αισθητήρα.

2. Αρχικοποίηση της συσκευής:

- Δεσμεύει και προετοιμάζει μια ιδιωτική δομή δεδομένων για την παρακολούθηση της κατάστασης του αισθητήρα.
- Συνδέει την ιδιωτική αυτή δομή με το αρχείο που άνοιξε ο χρήστης.

Η συνάρτηση επιστρέφει:

- 0: Αν η λειτουργία ήταν επιτυχής.
- **Αρνητικό κωδικό:** Αν υπήρξε αποτυχία (π.χ., `-ENODEV` ή `-ENOMEM`).

Ανάλυση Κώδικα

Αρχικά, εξαγάγουμε τον `minor` αριθμό από το `inode` του αρχείου που σχετίζεται με την συσκευή μέσω της συνάρτησης `iminor(inode)`. Αυτός ο αριθμός χρησιμοποιείται για να αναγνωρίσουμε τον αριθμό του αισθητήρα και το είδος του αισθητήρα. Αυτό ορίζεται στο `mk-lunix-devs.sh` script, όπως φαίνεται παρακάτω:

```
# Linux:TNG nodes: 16 sensors, each has 3 nodes.
for sensor in $(seq 0 15); do
    mknod /dev/lunix$sensor-batt c 60 [$sensor * 8 + 0]
    mknod /dev/lunix$sensor-temp c 60 [$sensor * 8 + 1]
    mknod /dev/lunix$sensor-light c 60 [$sensor * 8 + 2]
done
```

Συνεπώς, για:

1. **Αναγνώριση του αισθητήρα** (`sensor_nb`): Υπολογίζεται με την πράξη `bitwise minor » 3` (ισοδύναμο με ακέραια διαίρεση δια 8).
2. **Καθορισμό τύπου μέτρησης** (`sensor_type`): Υπολογίζεται με την πράξη `minor & 0x7` (ισοδύναμο με `minor % 8`).

Στη συνέχεια, αρχικοποιούμε την ιδιωτική δομή `linux_chrdev_state_struct` που θα περάσουμε στο `private_data` πεδίου του `file struct`. Η μνήμη για τη δομή `p_state` δεσμεύεται δυναμικά με την `kzalloc`. Αν η μνήμη δεν είναι διαθέσιμη, επιστρέφεται `-ENOMEM`.

Τα πεδία της δομής αρχικοποιούνται:

- `type`: Ορίζεται στον τύπο μέτρησης (`sensor_type`).
- `sensor`: Δείκτης στον συγκεκριμένο αισθητήρα.
- `buf_timestamp`, `buf_data` και `buf_lim`: Αρχικοποιούνται σε 0, σε μηδενική τιμή και 0 αντίστοιχα.

Έπειτα, συγχρονίζουμε πρόσβαση στην ιδιωτική μας δομή που είναι αποθηκευμένα τα δεδομένα για να εξασφαλίσουμε ακεραιότητα. Αυτό το πετυχαίνουμε με χρήση σεμαφόρων. Επιλέξαμε σημαφόρους αντί για spinlocks επειδή στην πρώτη περίπτωση η διεργασία που μπλοκάρει αφαιρείται από την ουρά εκτέλεσης της CPU μέχρι να της δοθεί πρόσβαση στο κρίσιμο τμήμα. Έτσι, δεν καταναλώνει πόρους της CPU (ενεργή αναμονή) όπως γίνεται με τα spinlocks. Αυτή η ιδιότητα κάνει το σημαφόρο ιδιαίτερα χρήσιμο σε καταστάσεις όπου περιμένουμε για κάποια εργασία εισόδου/εξόδου ή θέλουμε πρόσβαση σε δεδομένα τα οποία μπορεί να είναι κλειδωμένα για ώρα.

```
sema_init(&p_state->lock, 1);
```

Τέλος, περνάμε τα δεδομένα του `private_state` στο `private_data` field του `filp`. Το `filp` αποτελεί έναν δείκτη στη δομή του ανοικτού αρχείου που αντιπροσωπεύει το αρχείο στο οποίο γίνεται πρόσβαση. Το πεδίο `private_data` επιτρέπει στο driver να συσχετίσει μια προσαρμοσμένη δομή δεδομένων (στην προκειμένη περίπτωση, `p_state`) με το ανοικτό αρχείο. Έτσι, η ιδιωτική δομή `p_state` αποθηκεύεται στο πεδίο `private_data` του `filp`, καθιστώντας την προσβάσιμη σε άλλες λειτουργίες όπως το `read`.

Επιτυχής Ολοκλήρωση

Αν όλα τα βήματα ολοκληρωθούν επιτυχώς, η συνάρτηση επιστρέφει 0 και καταγράφεται σχετικό μήνυμα στο αρχείο καταγραφής (kernel log).

linux_chrdev_release

Η λειτουργία της συνάρτησης `linux_chrdev_release` ως file operation είναι η απελευθέρωση της μνήμης που έχει δεσμευτεί για το αρχείο που σχετίζεται με τη συσκευή μας. Καλώντας τη συνάρτηση `kfree` αποδεσμεύουμε το πεδίο `private_data`. Η συνάρτηση αυτή δεν αποτυγχάνει ποτέ και επιστρέφει πάντα 0.

```
static int linux_chrdev_release(struct inode *inode, struct file *filp)
{
    /*-----OUR CODE-----*/

    struct linux_chrdev_state_struct *private_state;

    private_state = filp->private_data;
```

```

    /* Clean up any resources associated with private_state if needed */
    kfree(private_state); // Free the allocated private state structure

    return 0;

    /*-----*/
}

```

linux_chrdev_read

Εισαγωγή

Η συνάρτηση `linux_chrdev_read` υλοποιεί τη λειτουργία ανάγνωσης για τον χαρακτήρα συσκευής του Linux. Η συνάρτηση καλείται όταν μία διεργασία κάνει `read()` στο αρχείο συσκευής που αντιστοιχεί σε κάποιον αισθητήρα του Linux. Η λειτουργία αυτή επιστρέφει δεδομένα αισθητήρα από την εσωτερική μνήμη του driver στον χώρο χρήστη.

Ρόλος της Συνάρτησης

Η `linux_chrdev_read`:

1. Ελέγχει αν υπάρχουν διαθέσιμα δεδομένα στον αισθητήρα.
2. Αναμένει νέα δεδομένα αν χρειάζεται (σε blocking λειτουργία).
3. Αντιγράφει τα δεδομένα στον buffer του χώρου χρήστη.
4. Ενημερώνει τη θέση του αρχείου (`f_pos`) για επόμενες αναγνώσεις.

Η συνάρτηση επιστρέφει:

- Τον αριθμό των bytes που διαβάστηκαν σε περίπτωση επιτυχίας.
- Αρνητική τιμή σε περίπτωση σφάλματος.

Παράμετροι

1. **struct file *filp:**
 - Αντιπροσωπεύει το ανοιχτό αρχείο συσκευής.
 - Περιέχει το πεδίο `private_data`, το οποίο δείχνει στη δομή κατάστασης (`linux_chrdev_state_struct`) που σχετίζεται με τη συγκεκριμένη διεργασία.
2. **char __user *usrbuf:**
 - Δείκτης στον buffer του χρήστη όπου τα δεδομένα θα μεταφερθούν.
3. **size_t cnt:**
 - Μέγιστος αριθμός bytes που ο χρήστης θέλει να διαβάσει.
4. **loff_t *f_pos:**
 - Δείκτης στη θέση αρχείου, η οποία ενημερώνεται για την παρακολούθηση της προόδου της ανάγνωσης.

Ανάλυση Κώδικα

Αρχικά, αποκτούμε πρόσβαση στο `private state` που πήραμε με χρήση σεμαφόρου. Αναλυτικότερα, Χρησιμοποιείται ο σεμαφόρος `state->lock` για να εξασφαλιστεί αποκλειστική πρόσβαση στη δομή κατάστασης. Παράλληλα, η συνάρτηση αυτή επιτρέπει να γίνει `interrupt` μέσω σήματος.

Έλεγχος για Αναγκαία Ενημέρωση Δεδομένων

Αν η θέση του αρχείου είναι 0, ελέγχεται αν απαιτείται ανανέωση δεδομένων. Συγκεκριμένα, όσο η συνάρτηση `linux_chrdev_state_update` επιστρέφει `-EAGAIN` (try again error code) συνεχίζουμε να περιμένουμε για νέα δεδομένα. Εφόσον, περιμένουμε για νέα δεδομένα, δεν είναι απαραίτητο να κρατάμε τον σεμαφόρο, καθώς η διεργασία απλώς περιμένει. Επομένως, ο σεμαφόρος απελευθερώνεται πριν μπει η διεργασία σε αναμονή (`wait_event_interruptible`) για να μην εμποδίζει άλλες διεργασίες να χρησιμοποιήσουν τους ίδιους πόρους.

```
up(&state->lock);
```

Θα ελέγξουμε τώρα το `mode` του αρχείου μας (Blocking, Non-Blocking). Αν το flag `O_NONBLOCK` είναι ενεργοποιημένο υποδεικνύει πως η διεργασία ζητά να διαβάσει δεδομένα, αλλά δεν περιμένει. Αν τα δεδομένα δεν είναι διαθέσιμα, η διεργασία επιστρέφει `-EAGAIN` και δοκιμάζει ξανά αργότερα. Ωστόσο εάν περιμένει για νέα δεδομένα τότε, μπαίνει σε μια ουρά αναμονής (`sensor->wq`, χρησιμοποιείται για να διαχειριστεί διεργασίες που περιμένουν να ικανοποιηθεί η συνθήκη.) και αναστέλλεται η διεργασία μέχρι να ικανοποιηθεί η συνθήκη `linux_chrdev_state_needs_refresh(state)`, δηλαδή να έχουμε νέα δεδομένα, ή να διακοπεί από ένα `signal`. Όταν γίνεται `update` ο `sensor` μέσω της `linux_sensor_update` στο `linux-sensor.c` αρχείο τότε καλείται η `wake_up_interruptible(&s->wq)` και ξυπνάει την διεργασία. Τέλος, ξανά κλειδώνουμε την δομή για να ξανά προσπαθήσουμε την παραπάνω διαδικασία. Σημειώνεται πως η δομή του κώδικα βασίζεται από το LDD3 βιβλίο, σελίδα 153.

Δήλωση EOF

Έπειτα, δηλώνουμε πως έχουμε φτάσει στο τέλος του αρχείου, EOF, βλέποντας το `buf_lim` πεδίο του `state`. Δηλαδή, εάν η τιμή του ισούται με 0 τότε δεν έχουμε κάτι άλλο να διαβάσουμε και 0 αντιπροσωπεύει τα bytes που διαβάστηκαν. Με λίγα λόγια, αν δεν υπάρχουν δεδομένα, επιστρέφεται 0 (End of File).

```
if (state->buf_lim == 0) { // Nothing to read, EOF reached
    ret = 0;
    goto out;
}
```

Αντιγραφή Δεδομένων στο Χρήστη

Ορίζουμε πόσα bytes θα διαβαστούν και θα αντιγραφούν στον buffer του χρήστη: Η μεταβλητή `cnt` αντιπροσωπεύει τον αριθμό των bytes που θέλουμε να διαβάσουμε (εάν είναι μεγαλύτερος από τον μέγιστο αριθμό που μπορούν να διαβαστούν τότε το `cnt` ισούται με αυτό) και στη συνέχεια ενημερώνουμε τον δείκτη του αρχείου `fpos` και θέτουμε την μεταβλητή επιστροφής ίση με τον αριθμό δεδομένων που διαβάσαμε.

```
*f_pos += cnt;
ret = cnt;
```

Αυτόματη Επανεκκίνηση Θέσης Αρχείου

Αν έχει διαβαστεί όλο το διαθέσιμο περιεχόμενο και ο δείκτης είναι ίσος ή μεγαλύτερος από τα διαθέσιμα δεδομένα, τότε το θέτουμε πάλι με 0 (τον φέρνουμε στην αρχή).

```
if (*f_pos >= state->buf_lim) {
    *f_pos = 0;
}
```

Τέλος, απελευθερώνουμε τον σεμαφόρο και επιστρέφουμε τα bytes που διαβάστηκαν.

```
up(&state->lock);
return ret;
```

linux_chrdev_init

Εισαγωγή

Η συνάρτηση `linux_chrdev_init` αποτελεί την αρχική συνάρτηση εκκίνησης (initialization function) για τον χαρακτήρα συσκευής (character device) του συστήματος Linux. Εκτελώντας την εντολή `insmod ./linux.ko` στο τερματικό καλείται η `module_init (linux_module_init)` από το `linux-module.c` αρχείο (που αποτελεί και το `main` αρχείο του οδηγού συσκευής μας) η οποία συνάρτηση `linux_module_init` καλεί την `linux_chrdev_init()`.

```

/*
 * Initialize the Linux character device
 */
if ((ret = linux_chrdev_init()) < 0)
    goto out_with_ldisc;

return 0;

```

Αυτό σημαίνει πως καλείται κατά την εκκίνηση του kernel module και είναι υπεύθυνη για την αρχική ρύθμιση και την καταχώρηση του χαρακτήρα συσκευής στο σύστημα.

Ρόλος της Συνάρτησης

Η συνάρτηση εκτελεί τις ακόλουθες ενέργειες:

1. Εγγράφει τον χαρακτήρα συσκευής στο υποσύστημα χαρακτήρων συσκευών του Linux.
2. Καθορίζει τα major και minor numbers συσκευής.
3. Δημιουργεί και συνδέει τους απαραίτητους χειριστές (handlers) που απαιτούνται για την πρόσβαση στη συσκευή.

Η συνάρτηση επιστρέφει:

- Μηδενικό (0) σε περίπτωση επιτυχίας.
- Αρνητική τιμή σε περίπτωση αποτυχίας, η οποία υποδεικνύει το είδος του σφάλματος.

Ανάλυση Κώδικα

Αρχικά, αρχικοποιούμε το character device, μέσω της συνάρτησης `cdev_init(&linux_chrdev_cdev, &linux_chrdev_fops);`
cdev_init:

- Αρχικοποιεί τη δομή **cdev**, η οποία αντιπροσωπεύει έναν χαρακτήρα συσκευής (είναι μια κεντρική δομή δεδομένων που χρησιμοποιεί ο kernel για να αναγνωρίζει και να διαχειρίζεται χαρακτήρες συσκευών).
- Παράμετροι:
 - `&linux_chrdev_cdev`: Δείκτης στη δομή `cdev` που αντιστοιχεί στη συσκευή. Ορίζεται ως Global Variable στο αρχείο `linux_chrdev.c`
 - `&linux_chrdev_fops`: Δείκτης στη δομή `file_operations`, η οποία περιέχει τις συναρτήσεις χειρισμού (handlers) για τη συσκευή, π.χ., `read`, `write`, `open`.

«Σημείωση»: Η δομή `file_operations` χρησιμοποιείται για να δηλώσει τις λειτουργίες που μπορούν να εκτελεστούν πάνω στη συσκευή. Είναι ένας πίνακας με δείκτες σε συναρτήσεις, όπου κάθε συνάρτηση αντιστοιχεί σε μια συγκεκριμένη λειτουργία.

Περιγραφή των μελών:

`.open`: Ορίζεται η συνάρτηση που καλείται όταν ανοίγει η συσκευή (`linux_chrdev_open`).
`.release`: Συνάρτηση που καλείται όταν κλείνει η συσκευή (`linux_chrdev_release`).
`.read`: Συνάρτηση για την ανάγνωση δεδομένων από τη συσκευή (`linux_chrdev_read`).

```

static struct file_operations linux_chrdev_fops =
{
    .owner          = THIS_MODULE,
    .open           = linux_chrdev_open ,
    .release        = linux_chrdev_release ,
    .read           = linux_chrdev_read ,
    .unlocked_ioctl = linux_chrdev_ioctl ,
    .mmap           = linux_chrdev_mmap
};

```

Στη συνέχεια, καθορίζουμε με την μακροεντολή `MKDEV` τον major και το minor number της συσκευής.

```
dev_no = MKDEV(LINUX_CHRDEV_MAJOR, 0);
```


- 'LINUX_CHRDEV_MAJOR': Προκαθορισμένος αριθμός που καθορίζει τον major αριθμό της συσκευής. Επόμενο βήμα αποτελεί η καταχώρηση της περιοχής των αριθμών συσκευής.

```
ret = register_chrdev_region(dev_no, linux_minor_cnt, "linux_chrdev");
```

register_chrdev_region:

- Καταχωρεί μια περιοχή αριθμών συσκευής (device numbers) για τον χαρακτήρα συσκευής.
- Παράμετροι:
 - dev_no: Ο πρώτος αριθμός συσκευής (κύριος και πρώτος δευτερεύων).
 - linux_minor_cnt: Ο συνολικός αριθμός των δευτερευόντων αριθμών συσκευής.
 - "linux_chrdev": Όνομα που θα χρησιμοποιείται για την αναφορά στη συσκευή.
- Αν αποτύχει, επιστρέφει αρνητική τιμή και εκτυπώνεται αντίστοιχο μήνυμα σφάλματος.

Στη συνέχεια, προσθέτουμε τη συσκευή χαρακτήρων στο σύστημα, συσχετίζοντάς την με τους παραπάνω καθορισμένους αριθμούς συσκευών.

```
ret = cdev_add(&linux_chrdev_cdev, dev_no, linux_minor_cnt);
```

cdev_add:

- Συνδέει τη δομή cdev με τους αριθμούς συσκευής που καταχωρήθηκαν προηγουμένως.
- Αν αποτύχει:
 - Εκτυπώνεται μήνυμα σφάλματος και αποδесμεύεται η περιοχή αριθμών συσκευής με unregister_chrdev_region.

Επιτυχής Αρχικοποίηση:

Επιστρέφεται μηδενική τιμή, και εμφανίζεται μήνυμα στο αρχείο καταγραφής (kernel log) για την επιτυχή αρχικοποίηση.

Σφάλματα:

Σε περίπτωση αποτυχίας, καθαρίζονται οι καταχωρήσεις που έχουν γίνει και επιστρέφεται αρνητικός κωδικός σφάλματος.